

Semi-Distributed Algebra Approaches for Solving KKT Systems in Model Predictive Control

March 10, 2025

Abstract

In this paper, we study distributed optimization for multi-agent systems, where agents collaborate to minimize a global objective function expressed as a sum of local objectives, subject to both local and globally coupled constraints. Motivated by applications across various disciplines, we examine semi-distributed algorithms where agents perform local computations based on their own information and interactions with a central coordinator. Specifically, we propose a semi-distributed optimization framework utilizing primal-dual interior-point and active-set methods to solve the resulting Karush-Kuhn-Tucker (KKT) systems. Numerical comparisons illustrate the performance improvements and computational advantages of our approach relative to conventional centralized or fully distributed methods.

1 Introduction

Quadratic Programming (QP) problems, characterized by quadratic objective functions and linear constraints, frequently arise across various control and optimization contexts. Notably, in linear-quadratic Model Predictive Control (MPC)—which optimizes control actions over a finite prediction horizon—each step can be transformed and solved as a convex QP. Rapid solution times for these problems are essential for real-time control applications, including autonomous driving and robotic systems, to ensure feasibility and performance.

Similarly, QP subproblems emerge when solving Nonlinear Programming (NLP) problems through Sequential Quadratic Programming (SQP). SQP iteratively approximates nonlinear optimization problems by solving a series of linear-quadratic approximate subproblems, where at each iteration approximate problem is formed by using the solution from previous iteration. Interior-point methods similarly address nonlinear problems through successive linear approximations. However, repeatedly solving

large-scale QPs in real-time poses significant computational challenges, especially in distributed control settings involving multi-agent systems, such as autonomous intersection management and coordinated robotic systems.

To meet these computational demands, distributed optimization techniques are increasingly employed. Distributed approaches decompose the KKT system and partition computations across multiple agents, allowing parallelization and reduced communication overhead by only communicating the required data rather than whole data set, thereby significantly improving scalability and computational speed. Common distributed frameworks include dual decomposition and Alternating Direction Method of Multipliers (ADMM). Although first order methods like ADMM are simple and robust, their slow convergence in certain scenarios motivates the exploration of faster approaches.

In this paper, we focus on developing and analyzing efficient semi-distributed optimization algorithms, specifically primal-dual interior-point and active-set methods for solving SQP which employ second order information, tailored to solve large-scale structured QPs arising from MPC and NLPs. We evaluate these algorithms through numerical experiments, demonstrating significant improvements in computational speed, scalability, and suitability for real-time distributed control applications, such as autonomous driving and intersection management systems.

2 Problem Formulation

Let us consider the NLP problem of the following form:

$$\min_x \sum_i f_i(x_i) \quad (1a)$$

$$\text{s.t. } g_i(x_i) = 0, \quad \forall i \quad (1b)$$

$$h_i^l \leq h_i(x_i) \leq h_i^u, \quad \forall i \quad (1c)$$

$$lb \leq \sum_i A_i x_i \leq ub, \quad \forall i \quad (1d)$$

where:

$f_i(x_i)$ s are scalar non-linear objective functions,
 $g_i(x_i)$ s, and $h_i(x_i)$ s are non-linear scalar constraint functions,

A_i s are linear coupling matrices.

Now if we employ SQP method to solve this problem. At iteration k , we start from the current iterate $x_i^{(k)}$, and define a local increment d_i such that:

$$x_i = x_i^{(k)} + d_i$$

SQP constructs a quadratic approximation of the objective and a linear approximation of the constraints around $x_i^{(k)}$ which results in the following system of equations,

$$\begin{aligned} \min_{d_i} \sum_i & \left[\frac{1}{2} d_i^T \nabla^2 f_i(x_i^{(k)}) d_i + \nabla f_i(x_i^{(k)})^T d_i \right] \\ \text{s.t. } & \nabla g_i(x_i^{(k)})^T d_i = -g_i(x_i^{(k)}), \\ & h_i^l - h_i(x_i^{(k)}) \leq \nabla h_i(x_i^{(k)})^T d_i \leq h_i^u - h_i(x_i^{(k)}), \\ & lb - \sum_i A_i x_i^{(k)} \leq \sum_i A_i d_i \leq ub - \sum_i A_i x_i^{(k)}, \end{aligned}$$

which is evidently a QP. The derivatives can be efficiently acquired using automatic differentiators such as CasADi, and then evaluated at each iteration. Next we will focus our attention to a generic QP problem as a NLP can be solved the same way after linearization.

3 Proposed Method

Consider following equation:

$$\min_x \sum_i \left(\frac{1}{2} x_i^T Q_i x_i + p_i^T x_i \right) \quad (2a)$$

$$\text{subject to: } G_i x_i = g_i, \quad \forall i \quad (2b)$$

$$h_i^l \leq H_i x_i \leq h_i^u, \quad \forall i \quad (2c)$$

$$lb \leq \sum_i A_i x_i \leq ub \quad (2d)$$

3.1 PD-IPM

First we shift our focus to PD-IPM algorithms, which solve constrained optimization problems by adding slack variables to inequalities and driving the smoothing (barrier) parameter to zero, in a successive sequence which converges to a local minimum of the problem. We briefly recall how PD-IPM will solve (2) and propose the distributed approach to solve computations of each iteration. We will use subscripts i for local agents where most of calculations will be done on and subscript c for a central agent, C , that has access to data from coupled constraint and will aggregate required information to solve the system and then broadcast to these so called local agents.

$$\min_{x,s} \sum_i \left(\frac{1}{2} x_i^T Q_i x_i + p_i^T x_i \right)$$

subject to:

$$G_i x_i = g_i, \quad \forall i$$

$$H_i x_i - s_i^l = h_i^l, \quad H_i x_i + s_i^u = h_i^u, \quad \forall i$$

$$\sum_i A_i x_i + s_c^l = lb, \quad \sum_i A_i x_i - s_c^u = ub$$

$$s_i^l \geq 0, \quad s_i^u \geq 0, \quad s_c^l \geq 0, \quad s_c^u \geq 0.$$

resulting in the following Lagrangian:

$$\begin{aligned} \mathcal{L} = & \sum_i \left(\frac{1}{2} x_i^T Q_i x_i + p_i^T x_i \right) + \sum_i \lambda_i^T (G_i x_i - g_i) \\ & + \sum_i \nu_i^l (H_i x_i - s_i^l - h_i^l) + \sum_i \nu_i^u (H_i x_i + s_i^u - h_i^u) \\ & + \mu^l \left(\sum_i A_i x_i - s_c^l - lb \right) + \mu^u \left(\sum_i A_i x_i + s_c^u - ub \right). \end{aligned}$$

With KKT condition:

$$\begin{aligned} \nabla_{x_i} \mathcal{L} = & Q_i x_i + p_i + G_i^T \lambda_i + H_i^T \nu_i^l + H_i^T \nu_i^u \\ & + A_i^T \mu^l + A_i^T \mu^u = 0 \quad \forall i, \end{aligned} \quad (3a)$$

$$\mu_i^l s_i^l = \tau \quad \forall i, \quad (3b)$$

$$\mu_i^u s_i^u = \tau \quad \forall i, \quad (3c)$$

$$\mu^l s_c^l = \tau, \quad (3d)$$

$$\mu^u s_c^u = \tau, \quad (3e)$$

$$\nu_i^l, \nu_i^u, s_i^l, s_i^u \geq 0, \quad (3f)$$

$$\nu_c^l, \nu_c^u, s_c^l, s_c^u \geq 0. \quad (3g)$$

Collecting primal-dual variables associated with each subscript in vectors z , $z_i = [x_i, \lambda_i, \nu_i^l, \nu_i^u, s_i^l, s_i^u]$ and $z_c = [\mu^l, \mu^u, s_c^l, s_c^u]$, we can also write equations (3b)-(3e) of the residuals $r_i(z_i) = 0$ and $r_c(z_c) = 0$.

Starting from initial guess $z_i^{[0]}$ and $z_c^{[0]}$ that satisfy condition on slack variables (3f)-(3g) strictly, the sequence of primal-dual solutions for each z_i and z_c is generated by the iteration

$$z^{[k+1]} = z^{[k]} + \alpha^{[k]} \Delta z^{[k]},$$

where k is the iteration index, $\alpha^{[k]}$ step size and $\Delta z^{[k]}$ the search direction obtained as solution for the KKT system,

$$M(Z^{[k]}) \Delta Z^{[k]} = -r_{\tau^{[k]}}(Z^{[k]})$$

with $\tau^{[k]}$ being the barrier parameter. The step size $\alpha^{[k]}$ is chosen such that updated solution satisfy (3f)-(3g) strictly while providing sufficient decrease for a merit function $\phi(z)$ and an update strategy for τ will ensure that $\tau^{[k]} \rightarrow 0$.

As our focus is to distribute the computations, we will perform most of computations required for solving the system and choosing step size locally. To this end we also rearrange the KKT residuals as, $r_i = [r_i^s, r_i^g, r_i^{hl}, r_i^{hu}, r_i^{sl}, r_i^{su}]$ and $r_c = [r_c^l, r_c^u, r_c^{sl}, r_c^{su}]$, whose equations are shown below.

$$\begin{aligned} r_i^s &= Q_i x_i + p_i + G_i^\top \lambda_i + H_i^\top \nu_i^l + H_i^\top \nu_i^u \\ &\quad + A_i^\top \mu^l + A_i^\top \mu^u \\ r_i^g &= G_i x_i - g_i \\ r_i^{hl} &= H_i x_i - s_i^l - h_i^l \\ r_i^{hu} &= H_i x_i + s_i^u - h_i^u \\ r_i^{sl} &= s_i^l \nu_i^l - \tau e \\ r_i^{su} &= s_i^u \nu_i^u - \tau e \end{aligned}$$

and

$$\begin{aligned} r_c^l &= \sum_i A_i x_i - s_c^l - lb \\ r_c^u &= \sum_i A_i x_i + s_c^u - ub \\ r_c^{sl} &= \text{diag}(s_c^l) \mu^l - \tau e \\ r_c^{su} &= \text{diag}(s_c^u) \mu^u - \tau e \end{aligned}$$

Using these notations for system of equations (3) we have the following linear matrix equations.

$$\begin{bmatrix} M_1 & 0 & 0 & \dots & 0 & C_1^\top \\ 0 & M_2 & 0 & \dots & 0 & C_2^\top \\ 0 & 0 & M_3 & \dots & 0 & C_3^\top \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & M_n & C_n^\top \\ C_1 & C_2 & C_3 & \dots & C_n & M_c \end{bmatrix} \begin{bmatrix} \Delta z_1 \\ \Delta z_2 \\ \Delta z_3 \\ \vdots \\ \Delta z_n \\ \Delta z_c \end{bmatrix} = - \begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ \vdots \\ r_n \\ r_c \end{bmatrix}$$

Where the matrices are as following

$$D_i = \begin{bmatrix} Q_i & G_i^\top & H_i^\top & H_i^\top & 0 & 0 \\ G_i & 0 & 0 & 0 & 0 & 0 \\ H_i & 0 & 0 & 0 & -I & 0 \\ H_i & 0 & 0 & 0 & 0 & I \\ 0 & 0 & \text{diag}(s_i^l) & 0 & \text{diag}(\nu_i^l) & 0 \\ 0 & 0 & 0 & \text{diag}(s_i^u) & 0 & \text{diag}(\nu_i^u) \end{bmatrix}$$

$$D_c = \begin{bmatrix} 0 & 0 & -I & 0 \\ 0 & 0 & 0 & I \\ \text{diag}(s^l) & 0 & \text{diag}(\mu^l) & 0 \\ 0 & \text{diag}(s^u) & 0 & \text{diag}(\mu^u) \end{bmatrix}$$

$$C_i = \begin{bmatrix} A_i & 0 & 0 & 0 & 0 & 0 \\ A_i & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The structure of this KKT matrix will allow us to distribute the algebra for solving this system of equations in the following way.

$$\begin{aligned} M_i \Delta z_i + C_i^T \Delta z_c &= -r_i \quad \forall i \\ \sum_i C_i \Delta z_i + M_c \Delta z_c &= -r_c \end{aligned}$$

Where we can locally solve for Δz_i , aggregate these values and solve for Δz_c as below.

$$\Delta z_i = -M_i^{-1} C_i^T \Delta z_c - M_i^{-1} r_i \quad (4)$$

$$\sum_i C_i (-M_i^{-1} C_i^T \Delta z_c - M_i^{-1} r_i) + M_c \Delta z_c = -r_c$$

$$(M_c - \sum_i C_i M_i^{-1} C_i^T) \Delta z_c = \sum_i C_i M_i^{-1} r_i - r_c \quad (5)$$

Let us denote the following matrices; $\mathcal{D}_i^C = C_i M_i^{-1} C_i^T$ and $\mathcal{D}_i^r = C_i M_i^{-1} r_i$. Following algorithms demonstrates our approach to solve this PD-IPM in distributed fashion.

To ensure that slack variables stay positive, we employ the following fraction from the boundary rule that is commonly used for all slack variables, $s_i^l, s_i^u, s_c^l, s_c^u$; choose an $\alpha \leq \alpha^{max}$:

$$s + \alpha^{max} \Delta s \geq \kappa s,$$

where κ is a small positive parameter. It is evident that each node (local and central) will find their respective α_i^{max} that satisfy this condition for all their relevant slack variables, i.e. (s_i^l, s_i^u) .

Algorithm 1 Distributed solution of PD-IPM KKT system

```

1: procedure SEARCHDIRECTION( $z, \tau$ )
2:    $\forall i$ : Compute  $\mathcal{D}_i^C$  and  $\mathcal{D}_i^r$  and pass to  $C$ 
3:    $C$ : Solve (5), pass  $\Delta z_c$  to all local node  $i$ -s
4:    $\forall i$  Solve (4) for  $\Delta z_i$ 
5: end procedure

```

We will also employ a l_1 merit function for guaranteeing an improvement in the solution by choosing the appropriate step length.

$$\phi_i(z_i) = \frac{1}{2}x_i^T Q_i x_i + p_i^T x_i + \sigma(\|G_i x_i - g_i\|_1 + \|H_i x_i - h_i^l - s_i^l\|_1 + \|H_i x_i - h_i^u + s_i^u\|_1)$$

and

$$\phi_c(z_c) = \sigma(\|\sum_i A_i x_i - lb - s_c^l\|_1 + \|H_i x_i - h_i^u + s_i^u\|_1)$$

and the merit function of the whole problem

$$\phi(z) = \sum_i \phi_i(z_i) + \phi_c(z_c)$$

We use the Armijo condition to choose an α that satisfied:

$$\phi(z + \alpha \Delta z) \leq \phi(z) + \alpha \gamma D\phi(z) \Delta z$$

where γ is a small positive number, and $D\phi(z)$ is the directional derivative of ϕ . We define $\Phi(\alpha) := \phi(z + \alpha \Delta z)$. The step size selection is summarized in Algorithm 2.

Algorithm 2 Distributed selection of step-size α

```

1: procedure STEPSIZESELECTION( $z, \Delta z, \tau$ )
2:    $\forall i$ : Compute and pass  $\alpha_i^{max}$ ,  $\Phi_i(0)$ ,  $D\Phi_i(0)$  to  $C$ 
3:    $C$ : Compute  $\alpha_c^{max}$ ,  $\Phi_c(0)$ ,  $D\Phi_c(0)$ 
4:    $C$ : Choose  $\alpha^{max}$ , assemble  $\Phi(0)$ ,  $D\Phi(0)$ 
5:   loop
6:      $C$ : Pass  $\alpha$  to all local node  $i$ 
7:      $\forall i$ : Compute and pass  $\Phi_i(\alpha)$  to  $C$ 
8:      $C$ : Compute  $\Phi_c(\alpha)$ , assemble  $\Phi(\alpha)$ 
9:     if  $\Phi(\alpha) < \Phi(0) + \alpha \gamma D\Phi(0)$  then
10:      return  $\alpha$  to all nodes  $i$ 
11:   else
12:      $\alpha \leftarrow \beta \alpha$ 
13:   end if
14: end loop
15: end procedure

```

The distributed PD-IPM algorithm using Algorithms 1 and 2 is summarized in Algorithm 3.

Algorithm 3 Basic Distributed PD-IPM

```

1: procedure QP PD-IPM SOLVER( $\tau^{[0]}$ )
2:    $C$ : Initialize  $z_c$  and send to all nodes  $i$ 
3:    $\forall i$ : Initialize  $z_i$ 
4:    $C$ : Choose  $\alpha^{max}$ , assemble  $\Phi(0)$ ,  $D\Phi(0)$ 
5:   loop
6:      $C$ : Send  $\tau$  to all local node  $i$ , form  $M_c, r_c$ 
7:      $\forall i$ : Form matrices  $M_i, r_i$ 
8:      $C$ : Compute  $\Phi_c(\alpha)$ , assemble  $\Phi(\alpha)$ 
9:      $\Delta z \leftarrow$  SearchDirection( $z, \tau$ )
10:     $\alpha \leftarrow$  StepSizeSelection( $z, \Delta z, \tau$ )
11:     $C$ : Update  $z_c \leftarrow z_c + \alpha \Delta z_c$ 
12:     $\forall i$ : Update  $z_i \leftarrow z_i + \alpha \Delta z_i$ 
13:    if TERMINATE( $r, \tau$ ) then
14:      return Solution found
15:    else
16:       $\tau \leftarrow$  UPDATEBARRIERPARAMETER( $\tau$ )
17:    end if
18:  end loop
19: end procedure

```

Termination criterion is whether the barrier parameter τ and the vector of residuals r have norms smaller a chosen small value ϵ satisfying desired accuracy. At the end of each loop of algorithm 3, barrier parameter is updated to ensure it is driven to 0.

3.2 Active set method

Next we will employ a parametric active set method, where the idea is to trace the solution of a linear homotopy parameterized by $\tau \in [0, 1]$ between a QP problem with a trivial or known solution (at $\tau = 0$) and the QP problem to be solved (at $\tau = 1$). The parametric problem to solve for recovering 2 is:

$$\min_{x(\tau)} \sum_i \left(\frac{1}{2} x_i(\tau)^T Q_i x_i(\tau) + p_i(\tau)^T x_i(\tau) \right) \quad (6a)$$

$$\text{subject to: } G_i x_i(\tau) = g_i(\tau), \quad \forall i \quad (6b)$$

$$h_i^l(\tau) \leq H_i x_i(\tau) \leq h_i^u(\tau), \quad \forall i \quad (6c)$$

$$lb(\tau) \leq \sum_i A_i x_i(\tau) \leq ub(\tau) \quad (6d)$$

If the gradient vector and bounds $(p_i(\tau), g_i(\tau), h_i^l, h_i^u, lb(\tau), ub(\tau))$ are taken as affine-linear functions of the homotopy parameter τ , the optimal solutions $x(\tau)$ will depend piecewise-linearly on τ . Additionally active set is constant on each linear segment for values of $\tau^{[n]}$ which are computed by feasibility tests. If we denote our optimal-dual

solution of decision and dual variables by $z(\tau)$, parametric active-set algorithm traces $z(\tau)$ by jumping from the beginning of one segment to the next, updating the working set $W(\tau)$ accordingly. We will denote by W_i and W_c vector of working set for local and central node respectively, with length equal to the number of inequalities and values for inequality j , $W_i(j) = -1$ if its lower active, $W_i(j) = 0$ if its inactive and $W_i(j) = 1$ if its upper active. In conjunction with this W , we will take the Lagrange multipliers of as negative. Here $p_i(\tau), g_i(\tau), h_i^l(\tau), h_i^u(\tau), lb(\tau), ub(\tau)$ are affine linear functions defined as $f(\tau) = (1 - \tau)f(0) + \tau f(1)$, where the values at $\tau = 1$ are taken such that we recover our original problem. If we know the active set for inequalities (6c) and for inequalities (6d) we will have the following conditions.

$$\begin{aligned} \nabla_{x_i} \mathcal{L} &= Q_i x_i(\tau) + p_i(\tau) - G_i^T \lambda_i(\tau) + H_i^T \nu_i^l \\ &\quad - H_i^T \nu_i^u + A_i^T \mu^l - A_i^T \mu^u = 0 \\ G_i x_i(\tau) - g_i &= 0, \\ (H_i x_i(\tau) - h_i^l(\tau)) \nu_i^l &= 0, \\ (H_i x_i(\tau) - h_i^u(\tau)) \nu_i^u &= 0, \\ (\sum_i A x_i(\tau) - lb(\tau)) \mu^l &= 0, \\ (\sum_i A x_i(\tau) - ub(\tau)) \mu^u &= 0, \end{aligned}$$

We will by ν_i the vector of dual variables for each local node containing values of Lagrange multipliers for upper active, lower active and inactive constraints $(\nu_i^l, \nu_i^u, 0)$. The vector of bounds of active constraints containing h_i^l, h_i^u that are active is denoted by h_i^W . The same notation for coupled constraint so that μ contains dual variables and a^W is the vector of bounds that are enforced. We can either hot-start this QP or start from zero for all variables that depend on parameter τ .

We will proceed to distribute the computations in similar fashion, by using the following notation.

$$\begin{aligned} z_i &= (x_i, -\lambda_i, -\nu_i^W) \\ \Delta z_i &= (\Delta x_i, -\Delta \lambda_i, -\Delta \nu_i^W) \\ r_i &= \begin{pmatrix} -(p_i(1) - p_i(\tau)) \\ g_i(1) - g_i(\tau) \\ h_i^W(1) - h_i^W(\tau) \end{pmatrix} \\ z_c &= -\mu^W \\ \Delta z_c &= -\Delta \mu^W \\ r_c &= a^W(1) - a^W(\tau). \end{aligned}$$

Then our system of equations to solve will be

$$\begin{bmatrix} M_1 & 0 & 0 & \dots & 0 & C_1^T \\ 0 & M_2 & 0 & \dots & 0 & C_2^T \\ 0 & 0 & M_3 & \dots & 0 & C_3^T \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & M_n & C_n^T \\ C_1 & C_2 & C_3 & \dots & C_n & 0 \end{bmatrix} \begin{bmatrix} \Delta z_1 \\ \Delta z_2 \\ \Delta z_3 \\ \vdots \\ \Delta z_n \\ \Delta z_c \end{bmatrix} = \begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ \vdots \\ r_n \\ r_c \end{bmatrix}$$

With aforementioned residual and vector of variables and matrices M_i that are

$$\begin{aligned} M_i &= \begin{bmatrix} Q_i & G_i^T & H_i^{WT} \\ G_i & 0 & 0 \\ H_i^W & 0 & 0 \end{bmatrix} \\ C_i &= [A_i^W \quad 0 \quad 0] \end{aligned}$$

We use the structure to solve as following

$$\begin{aligned} M_i \Delta z_i + C_i^T \Delta z_c &= r_i \quad \forall i \\ \sum_i C_i \Delta z_i &= r_c \end{aligned}$$

$$\Delta z_i = -M_i^{-1} C_i^T \Delta z_c + M_i^{-1} r_i \quad (7)$$

$$\sum_i C_i (-M_i^{-1} C_i^T \Delta z_c + M_i^{-1} r_i) = r_c$$

$$(\sum_i C_i M_i^{-1} C_i^T) \Delta z_c = r_c + \sum_i C_i M_i^{-1} r_i \quad (8)$$

Let us denote the following matrices; $\mathcal{D}_i^C = C_i M_i^{-1} C_i^T$ and $\mathcal{D}_i^r = C_i M_i^{-1} r_i$.

In choosing the homotopy step $\Delta \alpha$, we do it such that the condition $h_i^l(\tau) \leq H_i x_i(\tau) \leq h_i^u$ and $lb(\tau) \leq \sum_i A_i x_i(\tau) \leq ub(\tau)$ are satisfied at our next candidate point $x + \Delta \tau \Delta x$. Our homotopy step

Algorithm 4 Distributed solution of parametric active set KKT system

- 1: **procedure** SEARCHDIRECTION(z, τ)
 - 2: $\forall i$: Compute $\mathcal{D}_i^C$ and $\mathcal{D}_i^r$ and pass to C
 - 3: C : Solve (8), pass Δz_c , set $\Delta z_c = \frac{\Delta z_c}{1-\tau}$ to all local node i-s
 - 4: $\forall i$ Solve (7) for Δz_i , set $\Delta z_i = \frac{\Delta z_i}{1-\tau}$
 - 5: **end procedure**
-

Algorithm 5 Distributed selection of homotopy step length $\Delta\tau$

- 1: **procedure** STEPSIZESELECTION($z, \Delta z, \tau, W$)
 - 2: $\forall i$: Compute maximum $\Delta\tau$ satisfying primal and dual feasibility and pass to C
 - 3: C : Compute maximum $\Delta\tau$ satisfying primal and dual feasibility
 - 4: C : Choose biggest valid $\Delta\tau$
 - 5: C : Find index of blocking primal variable to activate the constraint or sign change in dual variable to deactivate
 - 6: **end procedure**
-

Algorithm 6 Basic Distributed parametric active set

- 1: **procedure** PQP SOLVER
 - 2: C : Initialize with $\tau = 0$
 - 3: $\forall i$: Initialize with $\tau = 0$
 - 4: **loop**
 - 5: $\forall i$: Form matrices M_i, r_i
 - 6: $\Delta z \leftarrow \text{SearchDirection}(z, \tau)$
 - 7: $\Delta\tau \leftarrow \text{StepSizeSelection}(z, \Delta z, \tau)$
 - 8: C : Update $z_c \leftarrow z_c + \Delta\tau\Delta z_c$ and change working set if needed
 - 9: $\forall i$: Update $z_i \leftarrow z_i + \Delta\tau\Delta z_i$ and change working set if needed
 - 10: **if** TERMINATE(r, τ) **then**
 - 11: **return** Solution found
 - 12: **else**
 - 13: $\tau \leftarrow \tau + \Delta\tau$
 - 14: **end if**
 - 15: **end loop**
 - 16: **end procedure**
-

is also limited by feasibility of dual variable, such that the sign of active constraint cannot change, i.e., $\lambda + \Delta\tau\Delta\lambda \leq 0$. Since we took dual variables of upper active constraints as negative and lower active ones as positive, we employ W in test for dual variables. For each local node we will have

$$\Delta\tau_i^p = \min\left\{\min\frac{H_i x_i - h_i^l(\tau)}{-H_i \Delta x_i}, \min\frac{h_i^u(\tau) - H_i x_i(\tau)}{H_i \Delta x_i}\right\}$$

$$\Delta\tau_i^d = \min\left(\frac{W(\tau) \circ \nu_i}{W(\tau) \circ \Delta\nu_i}\right)$$

where vector ν_i is comprised of ν_i^W for active constraints and filled with zero for the rest. We take $\Delta\tau_i = \min(\Delta\tau_i^p, \Delta\tau_i^d)$. Similarly for the coupled constraint we have

$$\Delta\tau_c^p = \min\left\{\min\frac{\sum_i A_i x_i - lb(\tau)}{-\sum_i A_i \Delta x_i}, \min\frac{ub(\tau) - \sum_i A_i x_i(\tau)}{\sum_i A_i \Delta x_i}\right\}$$

$$\Delta\tau_c^d = \min\left(\frac{W(\tau) \circ \mu}{W(\tau) \circ \Delta\mu}\right)$$

We take $\Delta\tau_c = \min(\Delta\tau_c^p, \Delta\tau_c^d)$. Our homotopy step τ will be the smallest of these $\Delta\tau$ s. We change active set of local or central node depending where this $\Delta\tau$ was found, and if its blocked by primal test we add that constraint to active set, and if there was a sign change we remove it from the active set.